시스템 프로그래밍

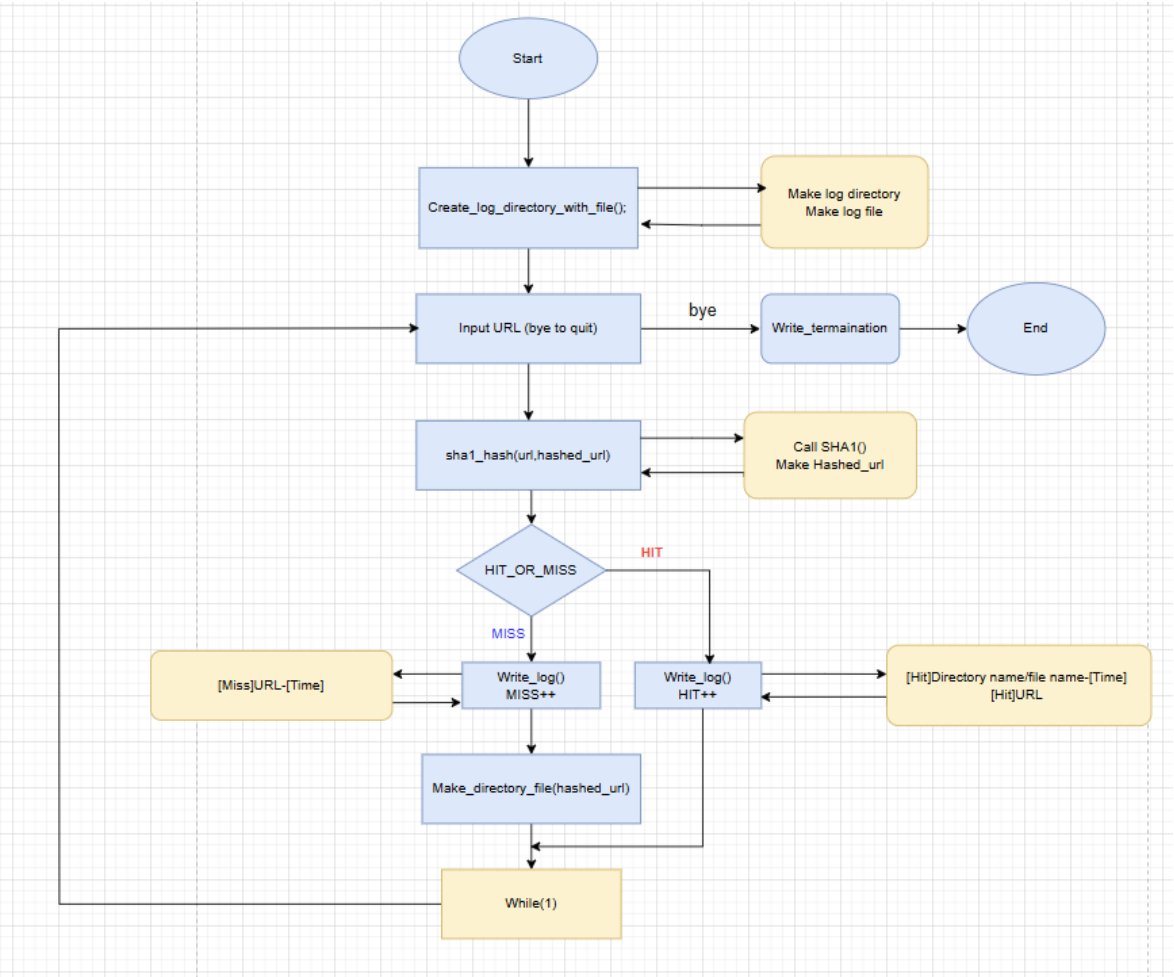
Proxy 1-2

강의 시간	목4 (B)
담당 교수님	최상호 교수님
학번	2021202089
이름	오나균

1) Introduction

이번 Proxy 1-2 과제는 이전에 구현한 Proxy 1-1의 코드에 이어서, 기능을 추가하는 것입니다. 우선 과제 설명을 보면, 기본적으로 Proxy 1-1에서 url을 입력받게해서, 해당 url을 해쉬화해서, 파일명을 "~/cache" 디렉토리에 저장하였는데, 이제 url을 입력받았을 때, 해당 url이 cache directory의 모든 파일의 파일명과 비교했을 때, hashed_URL과 동일한 파일명이 존재하지 않는다면 , MISS이고, 동일한 파일명이 존재한다면 HIT으로 판명해줍니다. 이제, 디렉토리 /logfile 과 /logfile/logfile.txt의 파일을 생성하여, logfile.txt에 앞에서 판명한 HIT, MISS에 대해서, HIT인 경우에는 Hashed_url명 – [처리된 시간기록], 그리고 원본 URL을 기록해줍니다. 그리고 MISS인 경우에는 원본 URL과 –[처리된 시간기록]을 기록해줍니다. 그리고 MISS된 URL에 대해서 당연히 이후에 hashed_url 처리를해서, ~/cache 디렉토리에 파일을 생성할 것입니다. 그리고 bye를 입력받으면, 프로그램이 종료되는데, 이때 Time() 함수를 사용하여, 프로그램의 실행시간을 기록해주고, 추가적으로 Hit과 Miss의 횟수도 기록해주며, 프로그램을 종료합니다.

2) Flow chart



3) Pseudo code

Proxy 1-1)

Make_directory_for_real(매개변수 : 만드려는 경로)

1. 경로(path)를 받아서 temp 배열에 저장한다.
2. 경로의 길이를 계산한다.
3. 경로의 마지막에 '/'가 있으면 제거한다.
4. temp 배열을 처음부터 끝까지 반복하면서: =>/ 를 만나면, 그 지점에서 temp 문자열을 끊어 중간 디렉토리를 만든다. => 중간 디렉토리를 만든 후, 다시 /를 복구하여 원래 경로를 유지한다.
5. 모든 중간 디렉토리를 만든 후, 최종 디렉토리를 만든다.

Make_directory_file(해쉬화된 경로)

1. home 디렉토리를 가져온다.
2. home 디렉토리에 /cache를 추가하여 기본 캐시 디렉토리 경로를 만든다.
3. 해시된 URL의 첫 번째, 두 번째, 세 번째 문자를 기반으로 하위 디렉토리 경로를 만든다.
4. 해당 디렉토리가 존재하지 않으면: => Make_directory_for_real() 함수를 사용하여 디렉토리를 만든다.
5. 해시된 URL의 나머지 부분을 사용하여 파일 경로를 만든다.
6. 해당 파일이 존재하지 않으면: a. fopen()을 사용하여 새 파일을 생성하고, 생성된 파일을 닫는다.

++ Proxy 1-2)

Create_log_directory_with_file()

1. 홈 디렉토리 경로 얻기
2. ~/logfile 디렉토리 경로 생성
3. 디렉토리가 존재하지 않으면

   3-1. 0777 권한으로 디렉토리 생성

4. ~/logfile/logfile.txt 파일 경로 생성

5. 파일이 존재하지 않으면

    5-1. 새 파일 생성 후 닫기

Write_log(URL, 해쉬화된 URL, Hit or Miss)

1. 홈 디렉토리 얻기

2. ~/logfile/logfile.txt 경로 생성

3. 현재 시간을 time_t 형식으로 얻기

    3.1 time_t를 struct tm (지역 시간)으로 변환 (localtime 사용)

    3.2 "YYYY/MM/DD, HH:MM:SS" 형식으로 문자열 생성

    3.3 생성된 문자열을 time_buf에 저장

4. 로그 파일을 append 모드로 열기

5. type이 "Hit"이면

    5-1. [Hit]디렉토리명/파일명-[시간] 작성

    5-2. [Hit]URL 작성

6. type이 "Miss"이면

    6-1. [Miss]URL-[시간] 작성

7. 파일 닫기

Write_termination(Hit 개수, Miss 개수, 실행 시간)

1. 홈 디렉토리 얻기
2. ~/home/log/logfile.txt 에 대한 경로 얻기
3. 얻은 경로를 통해, 실행시간, hit,miss에 대한 종료 로그 작성

HIT_OR_MISS(해쉬화된 URL)

1. hashed_url에서 앞 3글자를 추출 → 디렉토리명으로 사용
2. cache 디렉토리 경로 + 디렉토리명 → 전체 디렉토리 경로 생성
3. opendir()을 사용해 디렉토리 열기
    - 디렉토리를 열 수 없다면 (NULL 반환) → MISS (0 반환)
4. 디렉토리 내 파일 목록을 순회 (readdir 이용)
    - 각 파일 이름(dirent 구조체의 d_name)을 hashed_url의 뒷부분과 비교

- 일치하는 파일명이 있으면 → HIT (1 반환)

5. 끝까지 찾아도 없으면 → MISS (0 반환)

main()

1. 프로그램 시작 시간 저장

2. 무한 반복:

2-1. URL 입력 받기 (입력이 "bye"이면 반복 종료)

2-2. sha1_hash(url, hashed_url) 호출 → URL을 해시된 문자열로 변환

2-3. HIT_OR_MISS(hashed_url) 호출 → 결과를 result 변수에 저장

2-4. 만약 result == 1 (HIT):

- hit 카운트 증가

- Write_log("Hit", url, hashed_url) 호출 → HIT 로그 작성

2-5. 아니면 (MISS):

- miss 카운트 증가

- Write_log("Miss", url, NULL) 호출 → MISS 로그 작성

- Make_directory_file(hashed_url) 호출 → 디렉토리 및 파일 생성

3. 프로그램 종료 시간 저장

4. Write_termination() 호출 → 실행 시간, hit/miss 횟수 로그 작성

5. 프로그램 종료

4) **결과화면**

```
kw2021202089@ubuntu:~$ ./proxy_cache
input url>www.kw.ac.kr
input url>www.naver.com
input url>www.google.com
input url>www.kw.ac.kr
input url>www.naver.com
input url>klas.kw.ac.kr
input url>bye
```

⇨ 프로젝트 설명 예제와 같이 입력하였습니다.

```
kw2021202089@ubuntu:~$ ls -R ~/cache
/home/kw2021202089/cache:
3ef  d8b  e00  fed

/home/kw2021202089/cache/3ef:
9fd210fb8e00c8114ff978d282258ed8a48ea

/home/kw2021202089/cache/d8b:
99f68b208b5453b391cb0c6c3d6a9824f3c3a

/home/kw2021202089/cache/e00:
0f293fe62e97369e4b716bb3e78fababf8f90

/home/kw2021202089/cache/fed:
818da7395e30442b1dcf45c9b6669d1c0ff6b
```

⇨ ls -R ~/cache 명령어를 통해서, 현재 디렉토리부터, 모든 하위 디렉토리까지의 파일 출력

```
kw2021202089@ubuntu:~$ tree ~/cache
/home/kw2021202089/cache
├── 3ef
│   └── 9fd210fb8e00c8114ff978d282258ed8a48ea
├── d8b
│   └── 99f68b208b5453b391cb0c6c3d6a9824f3c3a
├── e00
│   └── 0f293fe62e97369e4b716bb3e78fababf8f90
└── fed
    └── 818da7395e30442b1dcf45c9b6669d1c0ff6b
```

⇨ tree ~/cache 명령어를 통해서, ~/cache 디렉토리와 파일들을 트리 형식으로 출력

```
kw2021202089@ubuntu:~$ cat ~/logfile/logfile.txt
[Miss]www.kw.ac.kr-[2025/4/10, 10:43:33]
[Miss]www.naver.com-[2025/4/10, 10:43:36]
[Miss]www.google.com-[2025/4/10, 10:43:38]
[Hit]e00/0f293fe62e97369e4b716bb3e78fababf8f90-[2025/4/10, 10:43:40]
[Hit]www.kw.ac.kr
[Hit]fed/818da7395e30442b1dcf45c9b6669d1c0ff6b-[2025/4/10, 10:43:42]
[Hit]www.naver.com
[Miss]klas.kw.ac.kr-[2025/4/10, 10:43:47]
[Terminated] run time: 18 sec. #request hit : 2, miss : 4
```

⇨ cat ~/logfile/logfile.txt 명령어를 통해서, logfile.txt 파일의 내용을 출력

⇨ 결과 화면을 통해, 프로젝트의 요구에 맞게 출력되는 것을 알 수 있습니다.

5) 고찰

이번 과제는 이전에 구현한 Proxy 1-1에 이어서, 새로운 기능을 구현하는 것이었는데, 우선 디렉토리 logfile 경로를 만들고, 그 디렉토리에 logfile.txt를 만드는 과정에서, 이를 함수를 통해 작성하였습니다.

이 함수를 만들어놓고, 이후에 logfile.txt에 로그를 작성하는 함수또한 구현하여, 코드를 테스트 해보았는데, logfile.txt에 아무것도 적히지 않아서, 뭐지 생각하다가 디버깅 과정에서, 보니까 log 디렉토리/ 파일 생성하는 함수에서, read, write, execute권한을 설정해놓지 않아서, 발생하는 문제였습니다. 저는 보통 버그가 발생할 때, 디버깅 기능을 사용하여,한줄 한줄 실행해보는 키로 디버깅을 하는데, 리눅스에서는 이것을 할 수 있는지 잘 몰라서 , 일단 make 한뒤 나오는 에러에 대한 메시지를 보고 디버깅을 할 수 있었습니다. (에러 메시지가 친절하게 나오는 거 같긴 합니다.)

이번에 발생한 에러는 프로젝트가 단순한 학습용이라, 간단하게 해결할 수 있었지만, 만약 나중에 회사에 가서 프로젝트를 해서, 이런 버그가 발생하는 지점을 알려면 쉽지 않을 것 같다는 생각이 들었습니다. 그래서 찾아보니까, 리눅스에 Vscode를 설치한 뒤에 gdb 기능을 연동하여서, 이를 통해 디버깅 기능을 쓸 수 있는 것을 알게되었습니다. 다음 과제부터는 이 기능을 유용하게 쓸 수 있을 것 같습니다.

6) Reference

https://yjcode.tistory.com/3 (리눅스 VScode)

https://mintnlatte.tistory.com/558 (dirent.h)

https://korbillgates.tistory.com/100 (time_t)